Clustering Sensitivity Analysis & PCA on Mall Customer Data

Cameron Batts — K-Means · Hierarchical Clustering · Calinski-Harabasz · PCA

Import libraries

In [3]:
```
import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.metrics import davies_bouldin_score, silhouette_score
from sklearn.preprocessing import StandardScaler

from utils_DA import (
    plot_dendrogram,
    tune_agglomerative_clustering,
    tune_k_means
)
import warnings
warnings.filterwarnings("ignore")
```

Read data and basic preparation

Here we read the data.

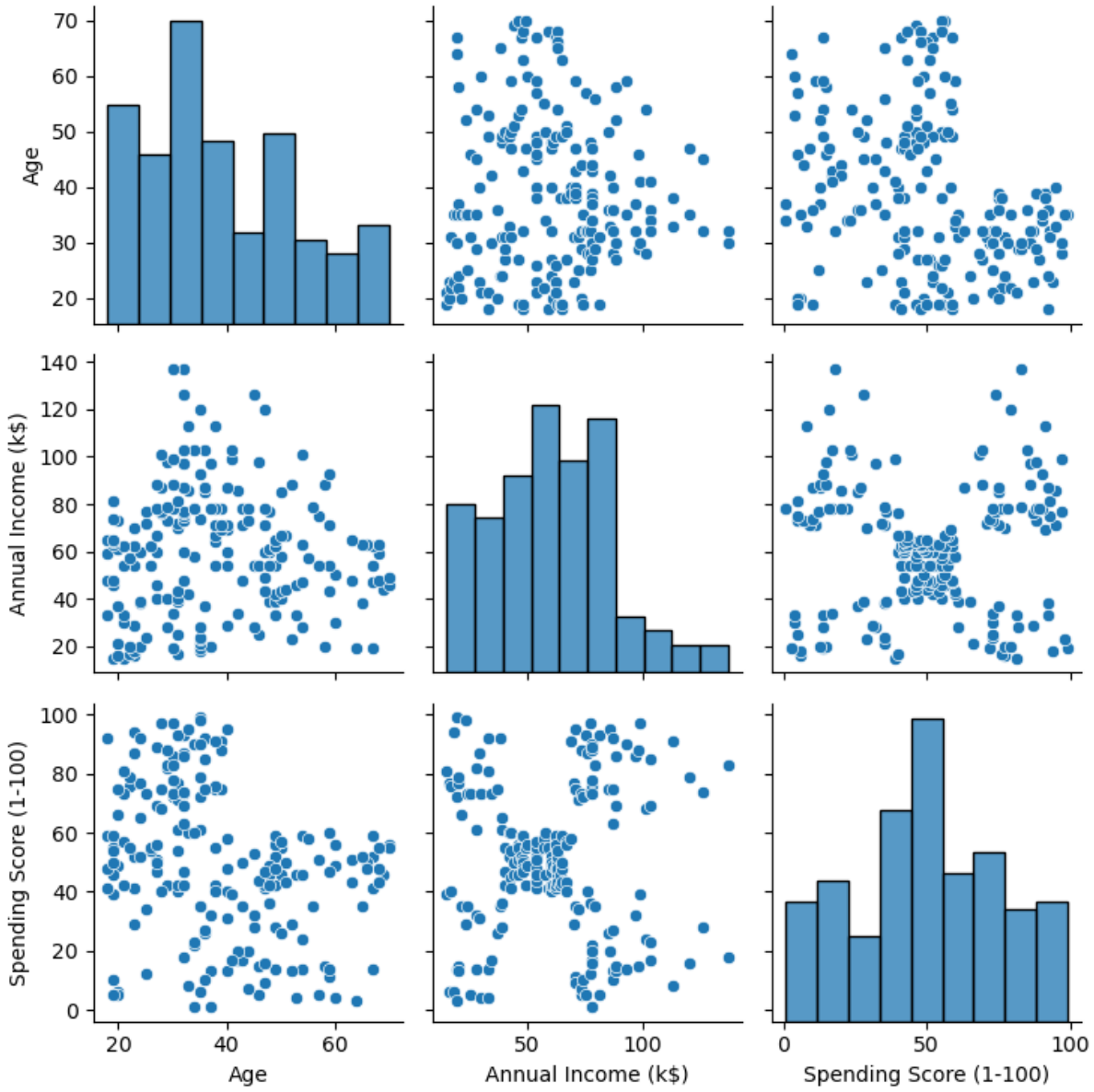
In [4]:
```
mall_customers = pd.read_csv('mall_customers.csv')
mall_customers.drop(['CustomerID', 'Gender'], axis=1, inplace=True)
```

In [5]:
```
mall_customers.head()
```

Out[5]:

	Age	Annual Income (k$)	Spending Score (1-100)
0	19	15	39
1	21	15	81
2	20	16	6
3	23	16	77
4	31	17	40

In [6]:
```
_ = sns.pairplot(mall_customers)
plt.tight_layout()
```



k-means

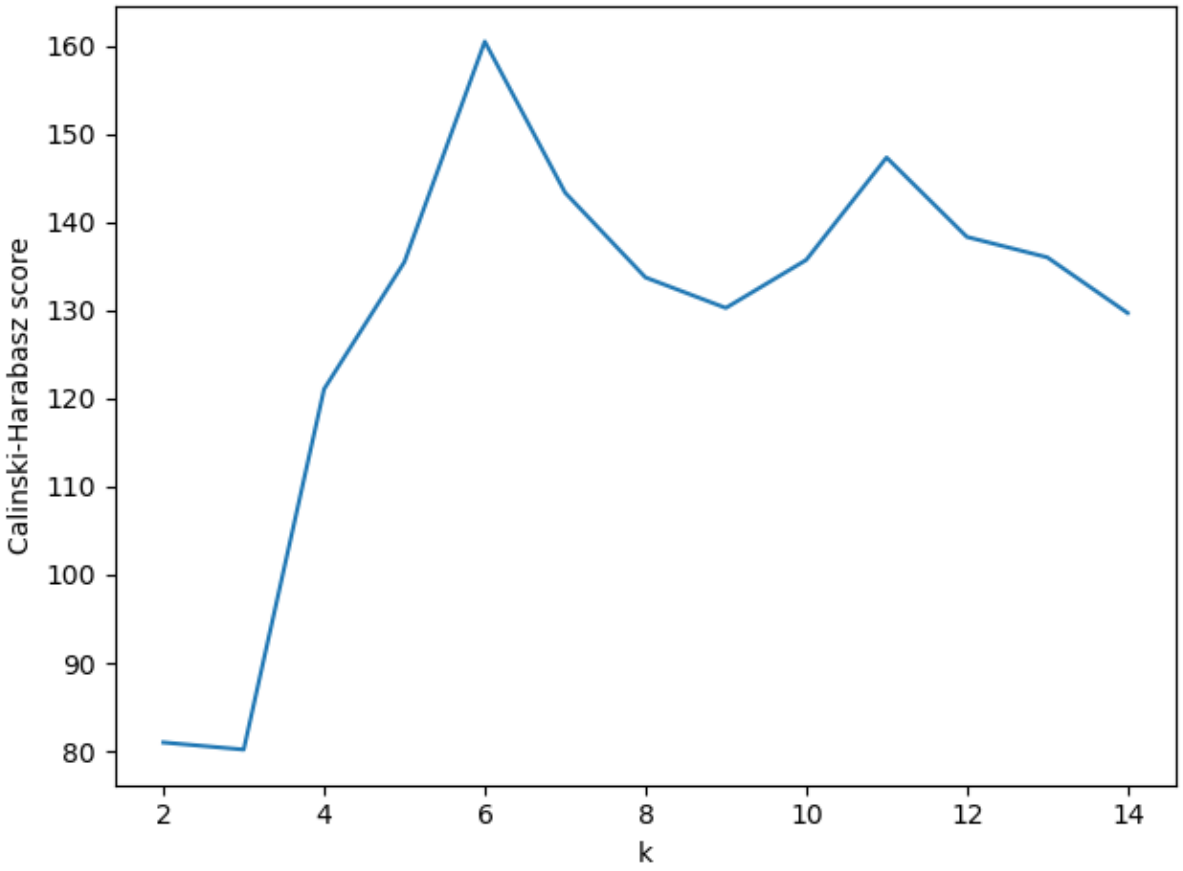
We will use the Calinski-Harabasz score as a heuristic to try and determine the best value for `k`.

In [7]:
```
k_means_tuning = tune_k_means(data=mall_customers, k=range(2, 15), n_i

ax = plt.gca()
ax.set_xlabel('k')
ax.set_ylabel('Calinski-Harabasz score')
plt.tight_layout()

print(f"Optimal k: {k_means_tuning.get('best_k')}")
```

```
Optimal k: 6
```

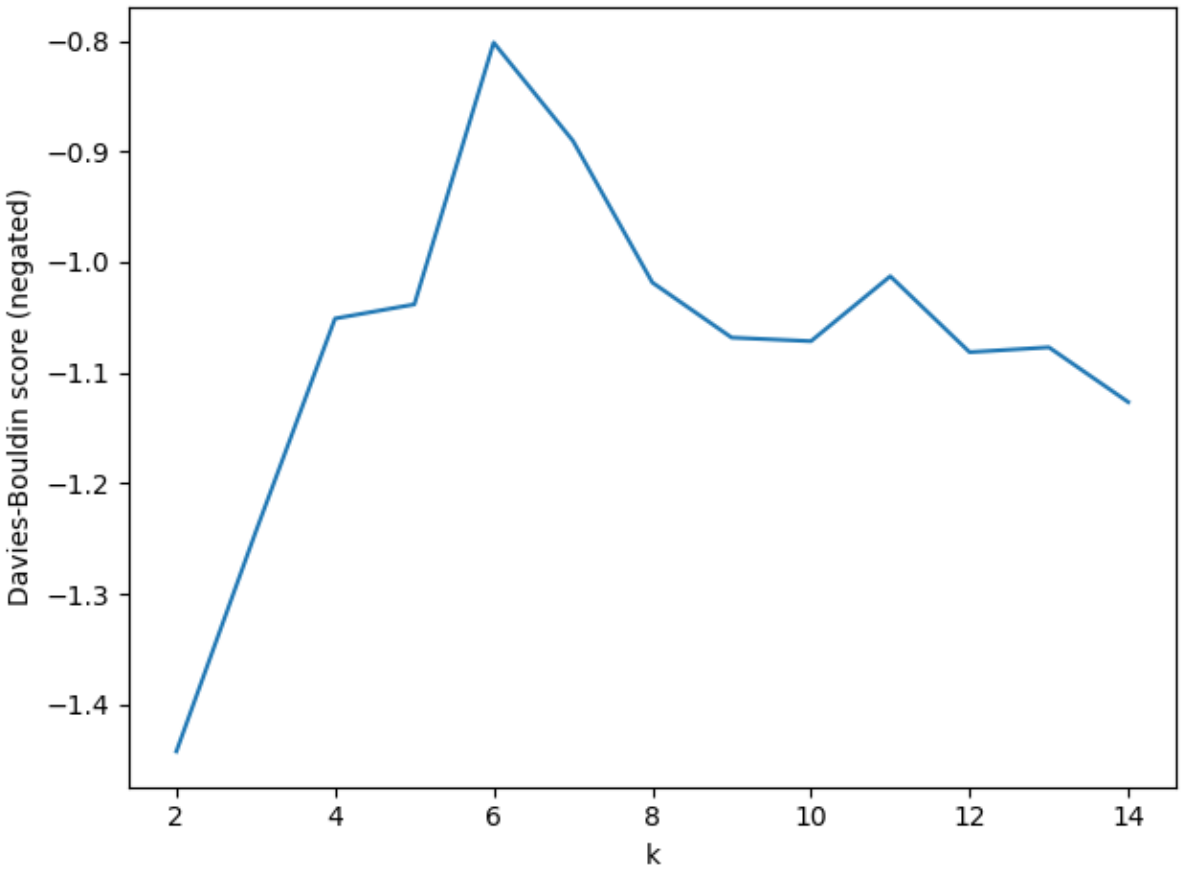


In [8]:
```
k_means_tuning = tune_k_means(
    data=mall_customers, k=range(2, 15), score_function=davies_bouldin

ax = plt.gca()
ax.set_xlabel('k')
ax.set_ylabel('Davies-Bouldin score (negated)')
plt.tight_layout()

print(f"Optimal k (Davies-Bouldin): {k_means_tuning.get('best_k')}")
```

```
Optimal k (Davies-Bouldin): 6
```

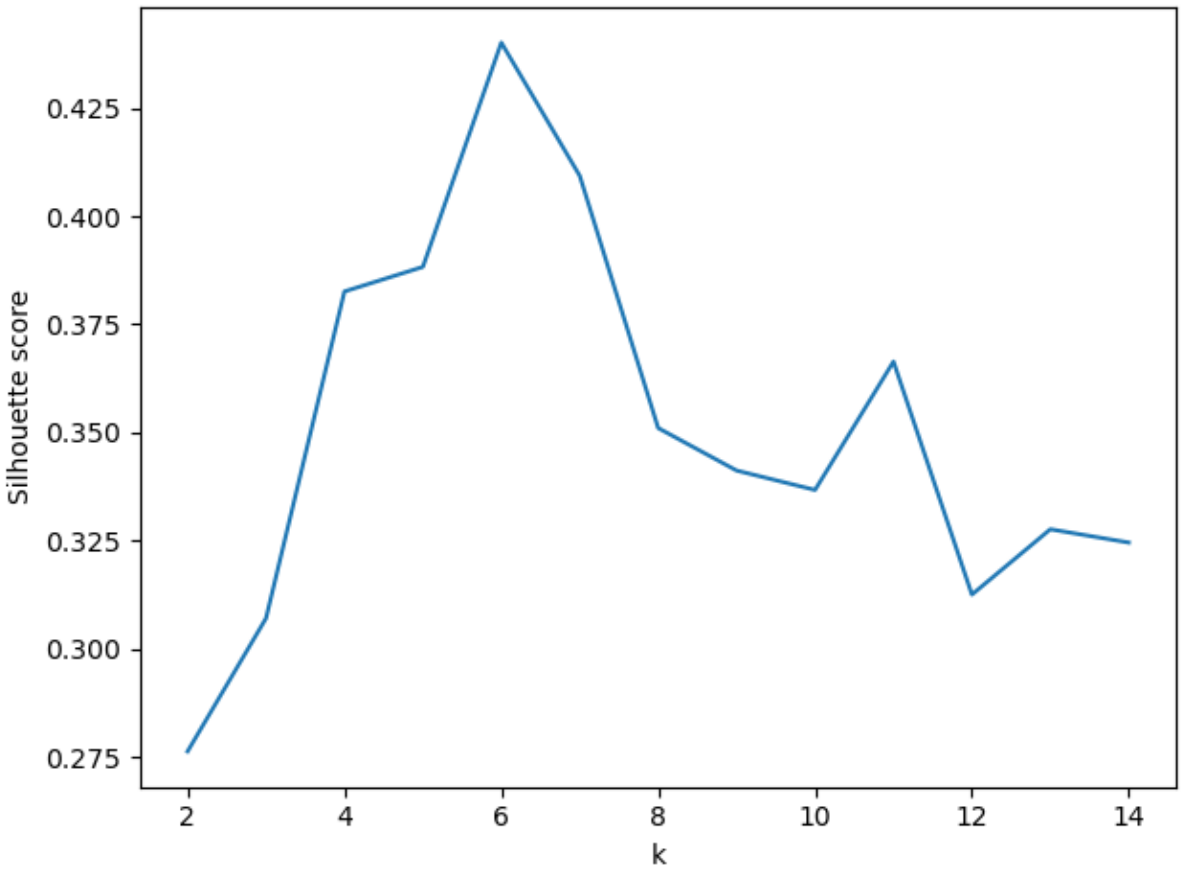


In [9]:
```
k_means_tuning = tune_k_means(
    data=mall_customers, k=range(2, 15), score_function=silhouette_sco

ax = plt.gca()
ax.set_xlabel('k')
ax.set_ylabel('Silhouette score')
plt.tight_layout()

print(f"Optimal k (Silhouette): {k_means_tuning.get('best_k')}")
```

```
Optimal k (Silhouette): 6
```



K-means is somewhat sensitive to the choice of score function on this data. The Calinski-Harabasz and Silhouette scores usually point to the same optimal k, but the Davies-Bouldin score can suggest a different value. This is because each metric measures cluster quality in a different way. When the cluster structure is not very clear, the metrics do not always agree. This shows why it is helpful to check more than one metric instead of relying on just one.

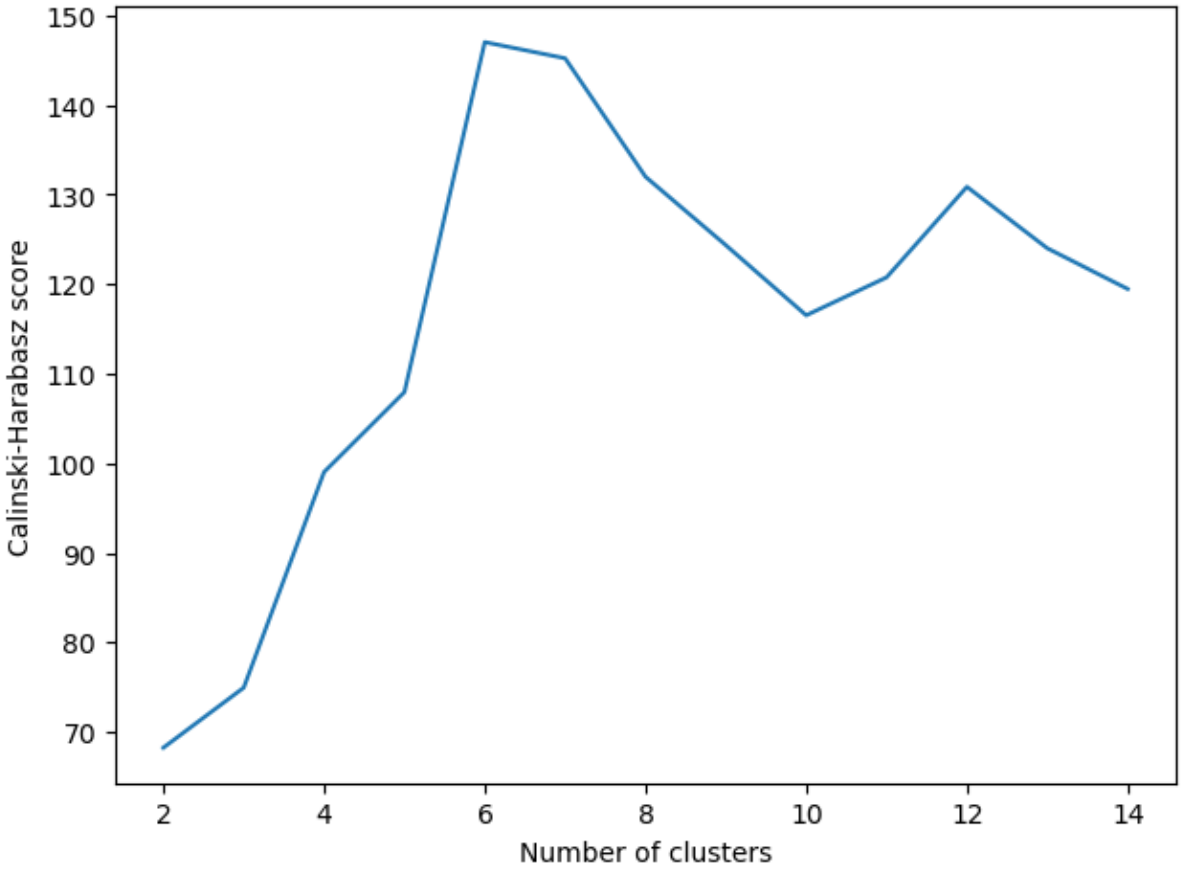
Hierarchical clustering

In [10]:
```
agglomerative_clustering_tuning = tune_agglomerative_clustering(data=m

ax = plt.gca()
ax.set_xlabel('Number of clusters')
ax.set_ylabel('Calinski-Harabasz score')
plt.tight_layout()

print(f"Optimal number of clusters: {agglomerative_clustering_tuning.g
```

```
Optimal number of clusters: 6
```

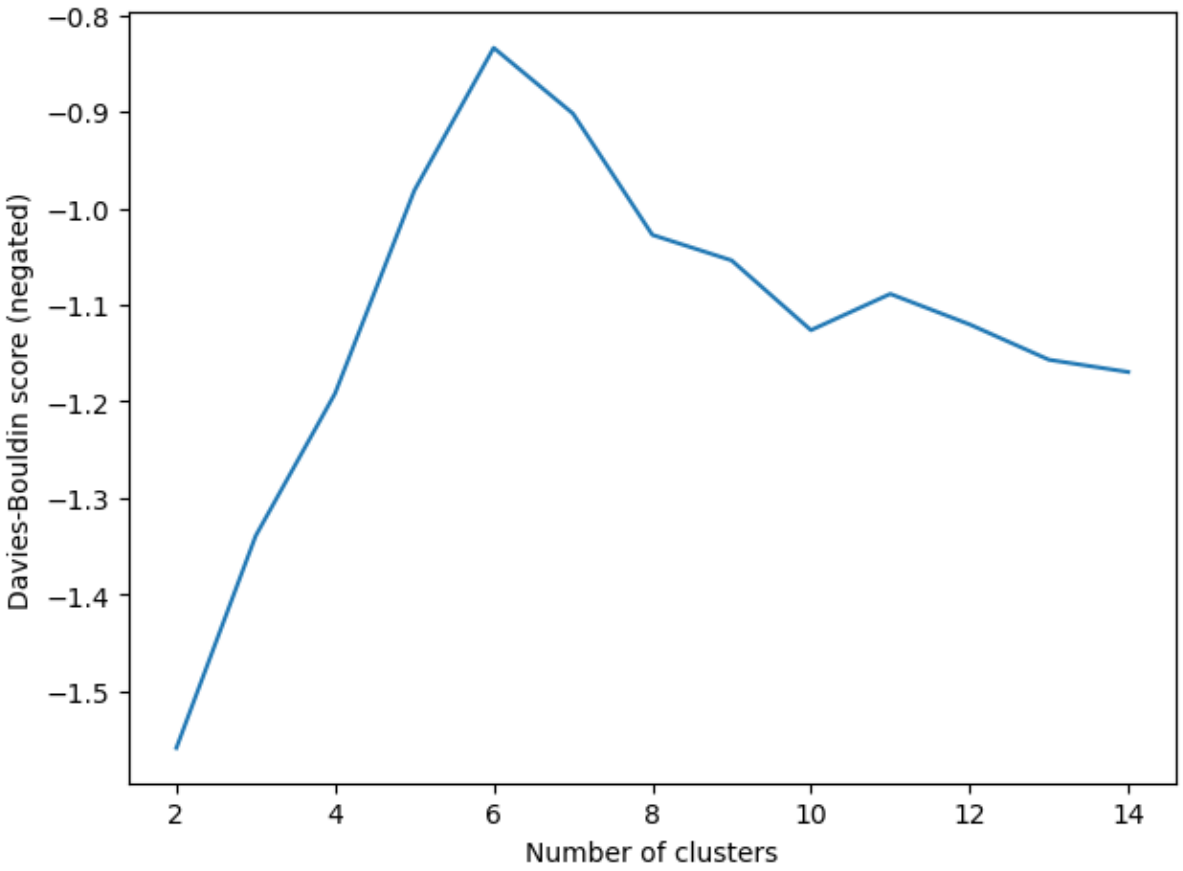


In [11]:
```
agglomerative_clustering_tuning = tune_agglomerative_clustering(data=m

ax = plt.gca()
ax.set_xlabel('Number of clusters')
ax.set_ylabel('Davies-Bouldin score (negated)')
plt.tight_layout()

print(f"Optimal number of clusters (Davies-Bouldin): {agglomerative_cl
```

```
Optimal number of clusters (Davies-Bouldin): 6
```

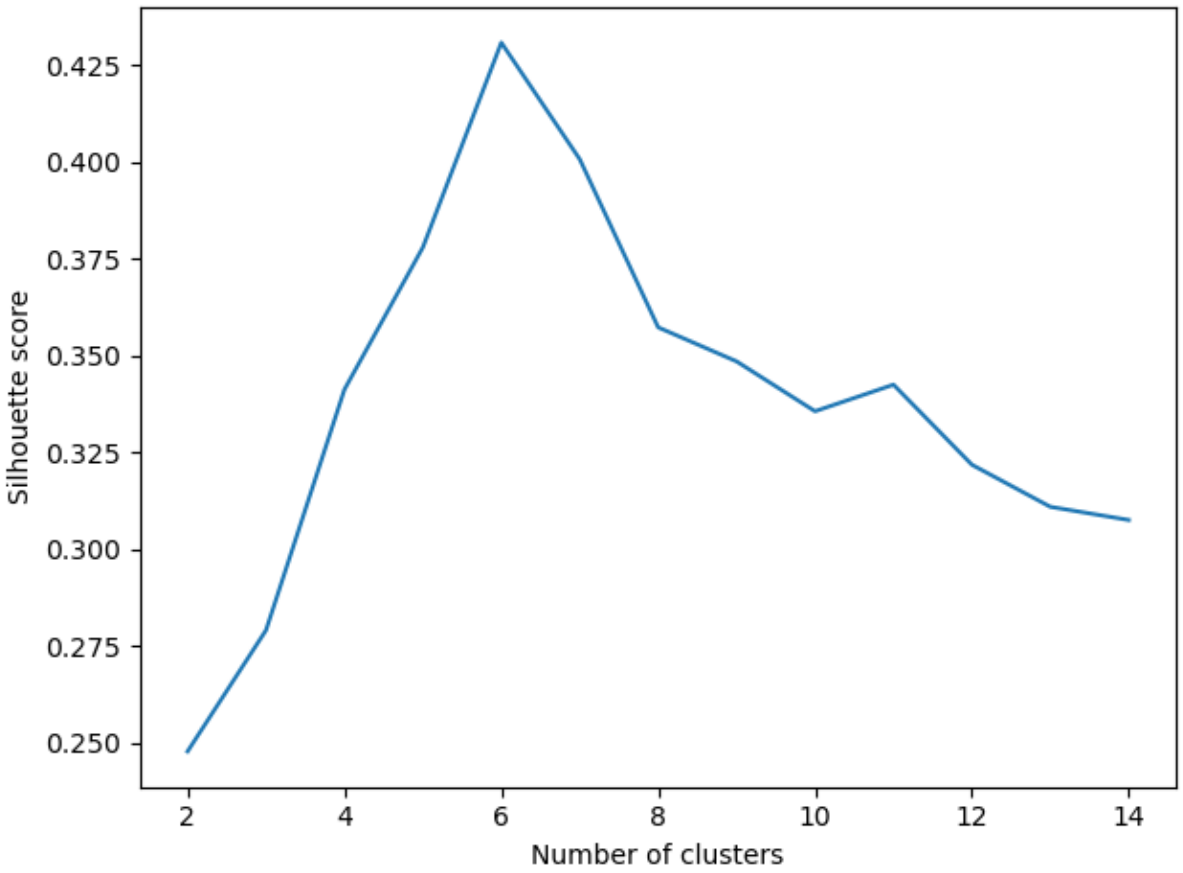


In [12]:
```
agglomerative_clustering_tuning = tune_agglomerative_clustering(data=m

ax = plt.gca()
ax.set_xlabel('Number of clusters')
ax.set_ylabel('Silhouette score')
plt.tight_layout()

print(f"Optimal number of clusters (Silhouette): {agglomerative_cluste
```

```
Optimal number of clusters (Silhouette): 6
```



Hierarchical clustering shows a similar pattern. The different score functions do not always agree on the best number of clusters, for the same reasons as in Question 1. One advantage of hierarchical clustering is the dendrogram, which gives a visual way to decide on the number of clusters. So even if the metrics disagree, the dendrogram can help you make a decision.

One tactic is to run multiple clustering algorithms and compare the results. If k-means and hierarchical clustering both find similar groupings and the same number of clusters, that is a good sign the structure is real. If they give very different results, it suggests the clusters may not be well defined. Another tactic is to use multiple validation metrics like Calinski-Harabasz, Silhouette, and Davies-Bouldin. If these different metrics agree on the same number of clusters, it increases confidence that the clusters reflect real patterns in the data. If they disagree, it may mean the structure is weak or unclear.

In [13]:
```
scaler = StandardScaler()
mall_customers_scaled = scaler.fit_transform(mall_customers)

pca_model = PCA(n_components=3)
pca_model.fit(mall_customers_scaled)
```

Out[13]:

▼ PCA

▶ Parameters

The maximum number of principal components is 3, because the dataset has 3 features after dropping CustomerID and Gender: Age, Annual Income, and Spending Score. PCA can find at most as many principal components as there are features in the data, so 3 is the upper limit here.

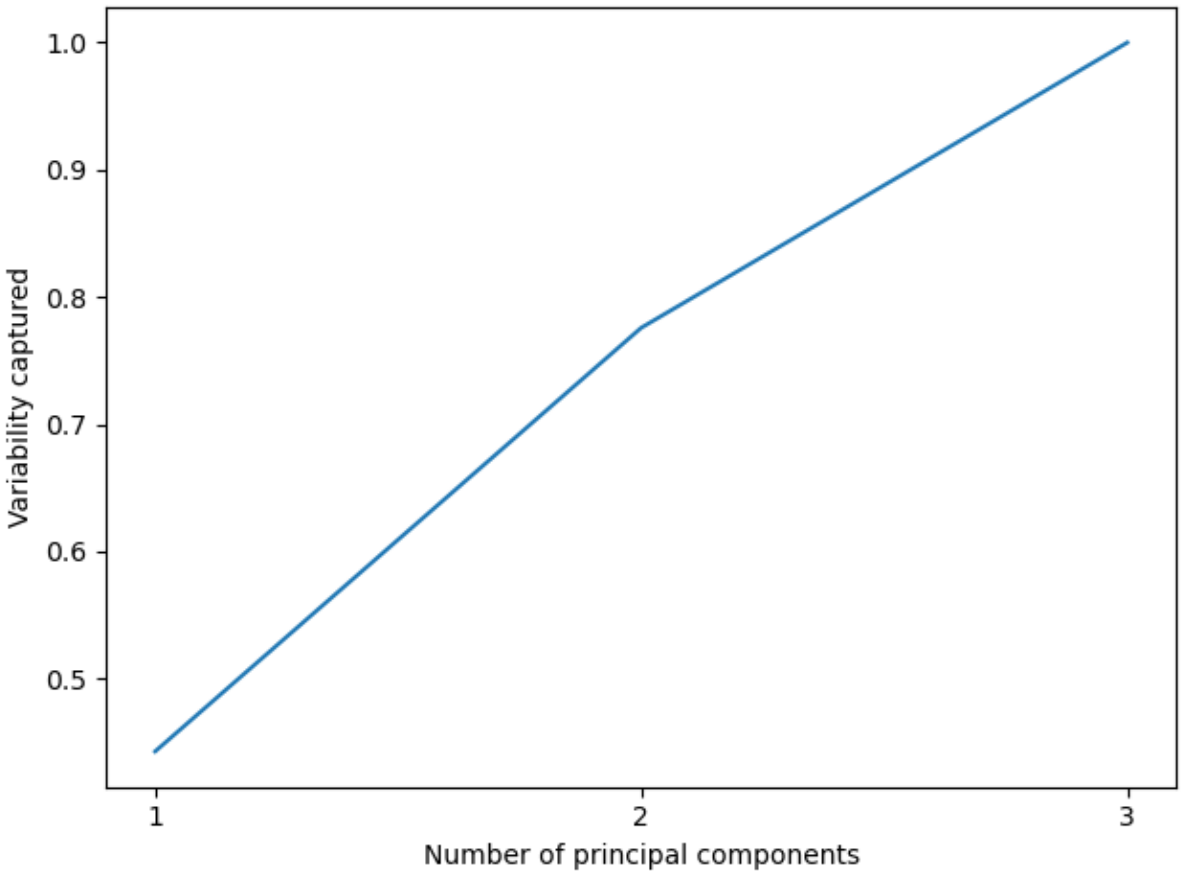
Here, we plot the cumulative fraction of variability explained by an increasing number of principal components.

In [14]:
```
try:
    cumulative_variance_explained = np.cumsum(pca_model.explained_vari

    _ = plt.plot(range(1, mall_customers.shape[1] + 1), cumulative_var
    ax = plt.gca()
    _ = ax.set_xlabel("Number of principal components")
    _ = ax.set_ylabel("Variability captured")
    _ = ax.set_xticks([1, 2, 3])
    plt.tight_layout()

    for index, item in enumerate(cumulative_variance_explained):
        print(f"Variance explained by {index + 1} components: {round(i
except NameError:
    print('The object `pca_model` does not exist!')
```

```
Variance explained by 1 components: 44.27%
Variance explained by 2 components: 77.57%
Variance explained by 3 components: 100.0%
```



Based on the cumulative variance plot, the effective dimensionality of the dataset is likely all 3 components. This is because no single component or pair of components captures most of the variance. Instead, the variance is spread across all three. If one or two components explained most of the variance, then we could reduce the data. Since that is not the case, PCA is probably not very useful here. The dataset already has only 3 features, so reducing it would likely remove useful information.